基于锁实现线程安全队列和栈容器

Posted on 2023-11-19 14:05:47 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2YNlWGTdr08l6DcfWeb6Rf7a7E5)

简介

本文介绍如何通过互斥锁和条件变量等并发机制实现线程安全的队列和栈容器。

线程安全的栈

实现一个线程安全的栈，我们能想到的是基于锁控制push和pop操作，比如下面的逻辑

```
#include <exception>
#include <mutex>
#include <stack>
#include <condition_variable>

struct empty_stack : std::exception
{
    const char* what() const throw();
};
template<typename T>
class threadsafe_stack
{
private:
    std::stack<T> data;
    mutable std::mutex m;
public:
    threadsafe_stack() {}

    threadsafe_stack(const threadsafe_stack& other)
    {
        std::lock_guard<std::mutex> lock(other.m);
        data = other.data;
    }

    threadsafe_stack& operator=(const threadsafe_stack&) = delete;

    void push(T new_value)
    {
        std::lock_guard<std::mutex> lock(m);
        data.push(std::move(new_value));    // <--- 1
    }

    std::shared_ptr<T> pop()
    {
        std::lock_guard<std::mutex> lock(m);
        if (data.empty()) throw empty_stack();  //  <--- 2
            std::shared_ptr<T> const res(
                std::make_shared<T>(std::move(data.top())));   // <--- 3
            data.pop();   // <--- 4
            return res;
    }

```

```
43.     void pop(T& value)
44.     {
45.         std::lock_guard<std::mutex> lock(m);
46.         if (data.empty()) throw empty_stack();
47.         value = std::move(data.top());   // <--- 5
48.             data.pop();   // <--- 6
49.     }
50.
51.     bool empty() const
52.     {
53.         std::lock_guard<std::mutex> lock(m);
54.         return data.empty();
55.     }
56. };
```

我们实现了push操作和pop操作

1. push操作里加锁，然后将数据通过 `std::move` 的方式移动放入stack中。我们思考如果1处因为机器内存不足push导致异常，此种情况并不会对栈已有的数据产生危险。

但是vector容器大家要考虑，因为vector存在内存不足时将数据拷贝转移到新空间的过程。那么对于vector这种动态扩容的容器该如何保证容器内数据在移动过程中出现了异常仍能不丢失呢?

我想到的一个方式就是管理vector的capacity，每次push的时候要判断一下vector的size和capacity是否相等，如果相等则手动扩容并将数据转移到新的vector，再释放旧有的vector。

但是同样会存在一个问题就是会造成内存溢出，因为vector的capacity会随着数据增加而增加，当vector中没有数据的时候capacity仍然很大。这种方式也可以通过swap的方式将当前大容量的vector和一个空的vector做交换，快速清空内存。这些操作和思路需结合实际开发情况而定。

1. pop提供了两个版本，一个是返回智能指针一个是返回bool类型，这两种我们分析，比如3处和4处也很可能因为内存不足导致构造智能指针失败，或者5处赋值失败，这种情况下抛出异常并不会影响栈内数据，因为程序没走到4和6处就抛出异常了。

2. pop函数内部判断栈是否空，如果为空则抛出异常，这种情况我们不能接受，异常是用来处理和预判突发情况的，对于一个栈为空这种常见现象，仅需根据返回之后判断为空再做尝试或放弃出栈即可。

为了解决栈为空就抛出异常的问题，我们可以做如下优化

```
1. template<typename  T>
2. class threadsafe_stack_waitable
3. {
4. private:
5.     std::stack<T> data;
6.     mutable std::mutex m;
7.     std::condition_variable cv;
8. public:
9.     threadsafe_stack_waitable() {}
10.
11.    threadsafe_stack_waitable(const threadsafe_stack_waitable& other)
12.    {
13.        std::lock_guard<std::mutex> lock(other.m);
14.        data = other.data;
15.    }
16.
17.    threadsafe_stack_waitable& operator=(const threadsafe_stack_waitable&) = delete;
18.
19.    void push(T new_value)
20.    {
21.        std::lock_guard<std::mutex> lock(m);
22.        data.push(std::move(new_value));    // <--- 1
23.        cv.notify_one();
24.    }
25.
26.    std::shared_ptr<T> wait_and_pop()
27.    {
28.        std::unique_lock<std::mutex> lock(m);
29.        cv.wait(lock, [this]()
30.            {
31.                if(data.empty())
32.                {
33.                    return false;
34.                }
35.                return true;
36.            }); //  <--- 2
37.
38.
39.        std::shared_ptr<T> const res(
40.            std::make_shared<T>(std::move(data.top())));    // <--- 3
41.        data.pop();   // <--- 4
42.        return res;
```

```
43.  }
44.
45.  void wait_and_pop(T& value)
46.  {
47.     std::unique_lock<std::mutex> lock(m);
48.     cv.wait(lock, [this]()
49.        {
50.           if (data.empty())
51.           {
52.              return false;
53.           }
54.           return true;
55.        });
56.
57.     value = std::move(data.top());   // <--- - 5
58.     data.pop();   // <--- - 6
59.  }
60.
61.  bool empty() const
62.  {
63.     std::lock_guard<std::mutex> lock(m);
64.     return data.empty();
65.  }
66.
67.  bool try_pop(T& value)
68.  {
69.     std::lock_guard<std::mutex> lock(m);
70.     if(data.empty())
71.     {
72.        return false;
73.     }
74.
75.     value = std::move(data.top());
76.     data.pop();
77.     return true;
78.  }
79.
80.  std::shared_ptr<T> try_pop()
81.  {
82.     std::lock_guard<std::mutex> lock(m);
83.     if(data.empty())
84.     {
```

```
85.             return std::shared_ptr<T>();
86.         }
87.
88.         std::shared_ptr<T> res(std::make_shared<T>(std::move(data.top())));
89.         data.pop();
90.         return res;
91.     }
92.
93. };
```

我们将pop优化为四个版本，四个版本又可以分为两个大类，两个大类分别为try_pop版本和wait_and_pop版本。

try_pop版本不阻塞等待队列有数据才返回，而是直接返回，try_pop又有两个版本，分别返回bool值和指针值。如果队列为空返回false或者空指针。

wait_and_pop版本阻塞等待队列有数据才返回，同样有两个版本，分别返回bool值和指针值。

但是上面的代码我们分析，假设此时栈为空，有一个线程A从队列中消费数据，调用wait_and_pop挂起, 此时另一个线程B向栈中放入数据调用push操作，notify一个线程消费队列中的数据。

此时A从wait_and_pop唤醒，但是在执行3或者5处时，因为内存不足引发了异常，我们之前分析过，即使引发异常也不会影响到栈内数据，所以对于栈的数据来说是安全的，但是线程A异常后，其他线程无法从队列中消费数据，除非线程B再执行一次push。因为我们采用的是 `notify_one` 的方式，所以仅有一个线程被激活，如果被激活的线程异常了，就不能保证该数据被其他线程消费了，解决这个问题，可以采用几个方案。

1. `wai_and_pop` 失败的线程修复后再次取一次数据。
2. 将 `notify_one` 改为 `notify_all` ，这样能保证通知所有线程。但是 `notify_all` 将导致所有线程竞争，并不可取。
3. 我们可以通过栈存储智能指针的方式进行，因为智能指针在赋值的时候不会引发异常。

稍后我们提供的线程安全队列的版本使用的就是第三种优化。

线程安全队列

队列和栈最大的不同就是队列为先入先出，有了线程安全的栈的开发思路，我们很快实现一个支持线程安全的队列

```
#include <mutex>
#include <queue>

template<typename T>
class threadsafe_queue
{
private:

    mutable std::mutex mut;
    std::queue<T> data_queue;
    std::condition_variable data_cond;

public:
    threadsafe_queue()
    {}

    void push(T new_value)
    {
        std::lock_guard<std::mutex> lk(mut);
        data_queue.push(std::move(new_value));
        data_cond.notify_one();    //<--- - ①
    }

    void wait_and_pop(T& value)    //<--- - ②
    {
        std::unique_lock<std::mutex> lk(mut);
        data_cond.wait(lk, [this] {return !data_queue.empty(); });
        value = std::move(data_queue.front());
        data_queue.pop();
    }

    std::shared_ptr<T> wait_and_pop()   // <--- - ③
    {
        std::unique_lock<std::mutex> lk(mut);
        data_cond.wait(lk, [this] {return !data_queue.empty(); });   // <--- - ④
        std::shared_ptr<T> res(
            std::make_shared<T>(std::move(data_queue.front())));
        data_queue.pop();
        return res;
    }

    bool try_pop(T& value)
```

```
43.     {
44.         std::lock_guard<std::mutex> lk(mut);
45.         if (data_queue.empty())
46.             return false;
47.         value = std::move(data_queue.front());
48.         data_queue.pop();
49.         return true;
50.     }
51.
52.     std::shared_ptr<T> try_pop()
53.     {
54.         std::lock_guard<std::mutex> lk(mut);
55.         if (data_queue.empty())
56.             return std::shared_ptr<T>();    //<--- - ⑤
57.         std::shared_ptr<T> res(
58.             std::make_shared<T>(std::move(data_queue.front())));
59.         data_queue.pop();
60.         return res;
61.     }
62.
63.     bool empty() const
64.     {
65.         std::lock_guard<std::mutex> lk(mut);
66.         return data_queue.empty();
67.     }
68. };
```

关于异常情况的分析和栈一样，上面的队列版本不存在异常导致数据丢失的问题。但是同样面临线程执行 `wait_and_pop` 时如果出现了异常，导致数据被滞留在队列中，其他线程也无法被唤醒的情况。

为了解决这种情况，我们前文提到了采用智能指针的方式，重新实现一下线程安全队列

```
template<typename T>
class threadsafe_queue_ptr
{
private:
    mutable std::mutex mut;
    std::queue<std::shared_ptr<T>> data_queue;
    std::condition_variable data_cond;
public:
    threadsafe_queue_ptr()
    {}
    void wait_and_pop(T& value)
    {
        std::unique_lock<std::mutex> lk(mut);
        data_cond.wait(lk, [this] {return !data_queue.empty(); });
        value = std::move(*data_queue.front());    //<--- - 1
            data_queue.pop();
    }
    bool try_pop(T& value)
    {
        std::lock_guard<std::mutex> lk(mut);
        if (data_queue.empty())
            return false;
        value = std::move(*data_queue.front());   // <--- - 2
            data_queue.pop();
        return true;
    }
    std::shared_ptr<T> wait_and_pop()
    {
        std::unique_lock<std::mutex> lk(mut);
        data_cond.wait(lk, [this] {return !data_queue.empty(); });
        std::shared_ptr<T> res = data_queue.front();   // <--- - 3
            data_queue.pop();
        return res;
    }
    std::shared_ptr<T> try_pop()
    {
        std::lock_guard<std::mutex> lk(mut);
        if (data_queue.empty())
            return std::shared_ptr<T>();
        std::shared_ptr<T> res = data_queue.front();   // <--- - 4
            data_queue.pop();
        return res;
```

```
43.     }
44.     void push(T new_value)
45.     {
46.         std::shared_ptr<T> data(
47.             std::make_shared<T>(std::move(new_value)));   // <--- 5
48.             std::lock_guard<std::mutex> lk(mut);
49.         data_queue.push(data);
50.         data_cond.notify_one();
51.     }
52.     bool empty() const
53.     {
54.         std::lock_guard<std::mutex> lk(mut);
55.         return data_queue.empty();
56.     }
57. };
```

在5处，我们push数据时需要先构造智能指针，如果构造的过程失败了也就不会push到队列中，不会污染队列中的数据。

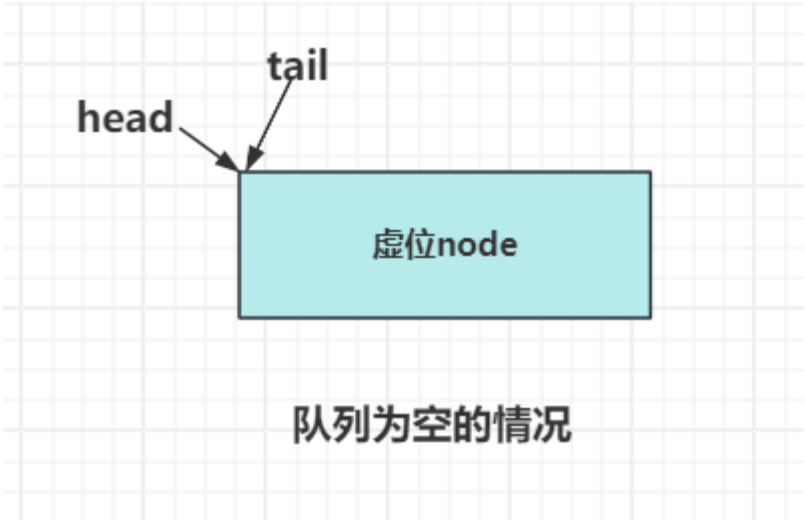
2，3处和4，5处我们仅仅时将智能指针取出来赋值给一个新的智能指针并返回。关于智能指针的赋值不会引发异常这一点在C++并发编程中提及，这一点我觉得有些存疑，我觉得书中表述的意思应该是指针在64位机器占用8个字节，所有智能指针共享引用计数所以在复制时仅为8字节开销，降低了内存消耗。

所以推荐大家存储数据放入容器中时尽量用智能指针，这样能保证复制和移动过程中开销较小，也可以实现一定意义的数据共享。

但是我们分析上面的代码，队列push和pop时采用的是一个mutex，导致push和pop等操作串行化，我们要考虑的是优化锁的精度，提高并发，那有什么办法吗?

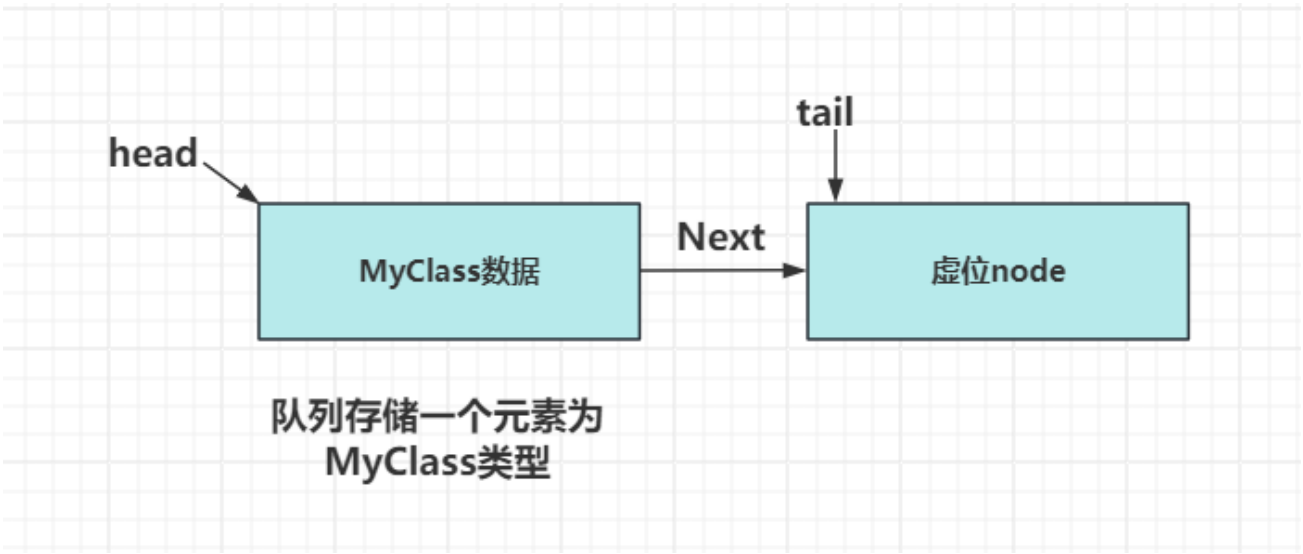
我们分析，队列和栈最本质的区别是队列是首尾操作。我们可以考虑将push和pop操作分化为分别对尾和对首部的操作。对首和尾分别用不同的互斥量管理就可以实现真正意义的并发了。

我们引入虚位节点的概念，表示一个空的节点，没有数据，是一个无效的节点，初始情况下，队列为空，head和tail节点都指向这个虚位节点。



当我们push一个数据，比如为MyClass类型的数据后，tail向后移动一个位置，并且仍旧指向这个虚位节点。

如下图



接下来我们实现这个版本的并发安全队列

```
template<typename T>
class threadsafe_queue_ht
{
private:
    struct node
    {
        std::shared_ptr<T> data;
        std::unique_ptr<node> next;
    };
    std::mutex head_mutex;
    std::unique_ptr<node> head;
    std::mutex tail_mutex;
    node* tail;
    std::condition_variable data_cond;

    node* get_tail()
    {
        std::lock_guard<std::mutex> tail_lock(tail_mutex);
        return tail;
    }
    std::unique_ptr<node> pop_head()
    {
        std::unique_ptr<node> old_head = std::move(head);
        head = std::move(old_head->next);
        return old_head;
    }
        std::unique_lock<std::mutex> wait_for_data()
    {
        std::unique_lock<std::mutex> head_lock(head_mutex);
        data_cond.wait(head_lock,[&] {return head.get() != get_tail(); }); //5
        return std::move(head_lock);
    }
        std::unique_ptr<node> wait_pop_head()
        {
            std::unique_lock<std::mutex> head_lock(wait_for_data());
                return pop_head();
        }
        std::unique_ptr<node> wait_pop_head(T& value)
        {
            std::unique_lock<std::mutex> head_lock(wait_for_data());
                value = std::move(*head->data);
            return pop_head();
```

```
43.     }
44.
45.
46.     std::unique_ptr<node> try_pop_head()
47.     {
48.         std::lock_guard<std::mutex> head_lock(head_mutex);
49.         if (head.get() == get_tail())
50.         {
51.             return std::unique_ptr<node>();
52.         }
53.         return pop_head();
54.     }
55.     std::unique_ptr<node> try_pop_head(T& value)
56.     {
57.         std::lock_guard<std::mutex> head_lock(head_mutex);
58.         if (head.get() == get_tail())
59.         {
60.             return std::unique_ptr<node>();
61.         }
62.         value = std::move(*head->data);
63.         return pop_head();
64.     }
65. public:
66.
67.   threadsafe_queue_ht() :  // <--- - 1
68.     head(new node), tail(head.get())
69.   {}
70.
71.   threadsafe_queue_ht(const threadsafe_queue_ht& other) = delete;
72.   threadsafe_queue_ht& operator=(const threadsafe_queue_ht& other) = delete;
73.
74.   std::shared_ptr<T> wait_and_pop() //  <------3
75.   {
76.     std::unique_ptr<node> const old_head = wait_pop_head();
77.     return old_head->data;
78.   }
79.
80.   void wait_and_pop(T& value)  //  <------4
81.   {
82.     std::unique_ptr<node> const old_head = wait_pop_head(value);
83.   }
84.
```

```
85.
86.     std::shared_ptr<T> try_pop()
87.     {
88.         std::unique_ptr<node> old_head = try_pop_head();
89.         return old_head ? old_head->data : std::shared_ptr<T>();
90.     }
91.     bool try_pop(T& value)
92.     {
93.         std::unique_ptr<node> const old_head = try_pop_head(value);
94.         return old_head;
95.     }
96.     bool empty()
97.     {
98.         std::lock_guard<std::mutex> head_lock(head_mutex);
99.         return (head.get() == get_tail());
100.    }
101.
102.    void push(T new_value)  //<------2
103.    {
104.        std::shared_ptr<T> new_data(
105.            std::make_shared<T>(std::move(new_value)));
106.        std::unique_ptr<node> p(new node);
107.        node* const new_tail = p.get();
108.        std::lock_guard<std::mutex> tail_lock(tail_mutex);
109.        tail->data = new_data;
110.        tail->next = std::move(p);
111.        tail = new_tail;
112.    }
113. };
```

node为节点类型，包含data和next两个成员。 data为智能指针类型存储T类型的数据。next为指向下一个节点的智能指针，以此形成链表。

上述代码我们的head是一个node类型的智能指针。而tail为node类型的普通指针，读者也可以用智能指针。

在1处构造函数那里，我们将head和tail初始指向的位置设置为虚位节点。

在2 处我们push数据的时候先构造T类型的智能指针存储数据new_data，然后我们构造了一个新的智能指针p, p取出裸指针就是新的虚位节点new_tail，我们将new_data赋值给现在的尾节点，并且让尾节点的next指针指向p, 然后将tail更新为我们新的虚位节点。

3，4处都是wait_and_pop的不同版本，内部调用了wait_pop_head，wait_pop_head内部先调用wait_for_data判断队列是否为空，这里判断是否为空主要是判断head是否指向虚位节点。如果不为空则返回unique_lock，我们显示的调用了move操作，返回unique_lock仍保留对互斥量的锁住状态。

回到wait_pop_head中，接下来执行pop_head将数据pop出来。

值得注意的是get_tail()返回tail节点，那么我们思考如果此时有多个线程push数据，tail节点已经变化了，我们此时在5处的判断可能是基于push之前的tail信息，但是不影响逻辑，因为如果head和tail相等则线程挂起，等待通知，如果不等则继续执行，push操作只会将tail向后移动不会导致逻辑问题。

pop_head中我们将head节点移动给一个old_head变量，然后将old_head的next节点更新为新的head。这里我觉得可以简化写为 `head=head->next` .

测试结果详见源码

总结

本文介绍了线程安全情况下栈和队列的实现方式

源码链接:

https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day14-ThreadSafeContainer (https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day14-ThreadSafeContainer)

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

浏览(1649) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(585) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)